

PROJECT REPORT ON COGNISCAN

(COMPUTER VISION-BASED DOCUMENT SCANNER)

Computer Vision — BYOP (Bring Your Own Project)

B.Tech CSE (Artificial Intelligence & Machine Learning) — 3rd Year

Submitted By : Aman Garg
Registration Number : 24BAI10826
Course Code : CSE3010

1. Introduction

The high-resolution cameras of smartphone has turned mobile device into portable digital scanners everyday. However, unprocessed photographs that are taken by human being are fundamentally different from flatbed scans. They suffer from perspective distortions, uneven lighting, and background environmental frame.

CogniScan is a computer vision application built in Python that automates the process of converting a raw, skewed photograph into a clean, professional, and cropped digital document. It utilizes classical image processing primitives to detect document boundaries, it mathematically handles the perspective, and enhance text legibility, entirely mimicking enterprise-level scanning software while executing locally without cloud dependencies.

2. Problem Statement

Digitizing physical documents using uncalibrated handheld cameras introduces several geometric and optical anomalies:

- **Perspective Skew:** Camera sensors are rarely perfectly parallel to the paper, resulting in trapezoidal distortion rather than a perfect rectangle.
- **Background Clutter:** Objects surrounding the document (e.g., tables, keyboards) are captured and must be isolated and removed.
- **Uneven Illumination:** Shadows cast by the user or ambient lighting create inconsistent contrast, rendering optical character recognition (OCR) systems ineffective and reducing human readability.
- **Privacy Concerns:** Many commercial mobile scanning applications require uploading sensitive documents to proprietary cloud servers for processing.

3. Functional Requirements

The system must successfully execute the following core operations:

1. **Image Loading:** Accept raw .jpg or .png image files via a command-line interface (CLI).
2. **Boundary Detection:** Automatically detect the four corners/edges of the primary document within the image frame easily.
3. **Perspective Rectification:** Transform the skewed document into a flat, top-down 2D perspective.
4. **Image Enhancement:** Provide selectable filtering modes:
 - *B&W Mode:* Remove shadows and binarizing the text for maximum contrast.
 - *Colour Mode:* Correct brightness while preserving original ink colors and stamps.

5. **Output Generation:** Save the cropped, processed image to a designated local directory.
6. **PDF Export:** Optionally compile the processed image into a standard .pdf document.

4. Non-functional Requirements

1. **Performance:** The core processing pipeline must execute in under 1 second on a standard CPU without requiring GPU acceleration.
2. **Portability:** The system must be cross-platform, running on Windows, macOS, or Linux via a standard Python environment.
3. **Privacy & Security:** 100% of the image processing must occur locally on the host machine. No external API calls or cloud storage can be utilized.
4. **Modularity:** The codebase must separate into distinct utility modules (preprocessing, transformations, enhancement) to allow for future ML integrations.

5. System Architecture

CogniScan operates on a deterministic, 6-step sequential computer vision pipeline:

1. **Grayscale & Blur:** The image is converted to a single-channel grayscale matrix and smoothed using a Gaussian Blur filter to suppress high-frequency background noise.
2. **Edge Detection:** A Canny Edge Detector identifies regions of high intensity gradients, isolating the structural borders of the paper.
3. **Hierarchical Contours:** The algorithm filters the edge map for closed polygons, isolating the largest valid 4-point contour as the target document.
4. **Projective Geometry:** A homography matrix is calculated from the detected corners to perform a perspective warp.
5. **Adaptive Thresholding:** The rectified image is processed using localized thresholding to eliminate shadows.
6. **I/O Operations:** The matrix is encoded back into an image file and optionally converted to PDF using binary serialization.

6. Design Decisions & Rationale

1. **OpenCV vs. Deep Learning:** A classical OpenCV pipeline (Canny + Contours) was chosen over a Deep Learning segmentation model (like U-Net) to ensure the application remains lightweight, CPU-friendly, and requires no heavy model weights or GPU configuration.

2. **CLI over GUI:** A Command Line Interface was prioritized to allow the tool to be easily integrated into larger automated shell scripts or batch processing workflows in the future.
3. **Adaptive vs. Global Thresholding:** Global thresholding fails under uneven lighting (e.g., half the page in shadow). Adaptive thresholding was implemented because it calculates the threshold for small, localized pixel regions independently, ensuring text in shadows remains legible.

7. Implementation Details

- **Language & Environment:** Python 3.11 virtual environment (venv).
- **Core Computer Vision:** `opencv-python` handles all matrix transformations, contour mapping, and thresholding.
- **Mathematical Operations:** `numpy` is utilized to calculate Euclidean distances between corner points to establish the dimensions of the warped bounding box.
- **PDF Compilation:** `img2pdf` and `Pillow` (PIL) are used to encode the optimized image binary directly into a standard PDF format without loss of resolution.
- **Execution Command Syntax:** `python cogniscan.py --input <path> --mode <bw/color/original> --pdf`

8. Screenshots / Results

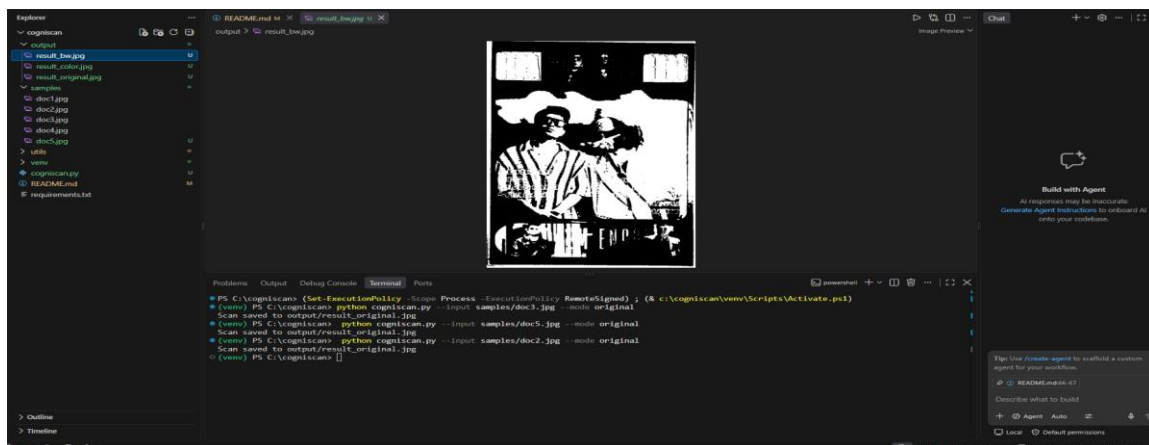
-- Unprocessed input image



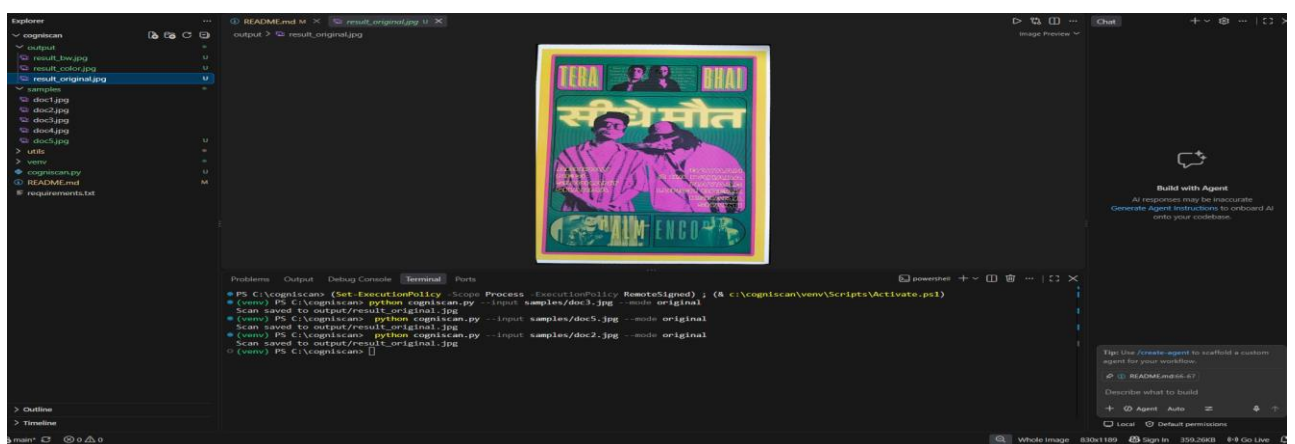
-- CLI Execution and output logging

```
Problems Output Debug Console Terminal Ports
PS C:\cogniscan> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned); (& c:\cogniscan\venv\Scripts\Activate.ps1)
(venv) PS C:\cogniscan> python cogniscan.py --input samples/doc3.jpg --mode original
Scan saved to output/result_original.jpg
(venv) PS C:\cogniscan> python cogniscan.py --input samples/doc5.jpg --mode original
Scan saved to output/result_original.jpg
(venv) PS C:\cogniscan> python cogniscan.py --input samples/doc2.jpg --mode original
Scan saved to output/result_original.jpg
(venv) PS C:\cogniscan>
```

-- Perspective-corrected B&W output



-- Corrected Color mode output



9. Testing Approach

1. **Unit Testing:** Each module in the `utils/` directory was tested independently. For example, the `transform.py` module was fed manual, hardcoded corner coordinates to verify the mathematical correctness of the homography warp before integrating it with the edge detector.
2. **Edge-Case Testing:** The system was evaluated against challenging inputs:
 - Documents placed on highly textured surfaces (wood, fabric).
 - Documents with bent corners.
 - Images captured under low-light conditions with heavy shadows.
3. **File System Testing:** Ensured the system handles missing directories gracefully (e.g., creating the `output/` folder dynamically if it does not exist at runtime).

10. Challenges Faced

1. **Low-Contrast Boundaries:** When a white document was placed on a white or light-grey table, the Canny edge detector struggled to define a clear boundary, causing the contour extraction to fail.
2. **Coordinate Ordering:** The perspective transform requires the four corner coordinates to be in a strict order (Top-Left, Top-Right, Bottom-Right, Bottom-Left). Writing the NumPy logic to consistently sort arbitrary contour points into this exact sequence required careful mathematical debugging.
3. **Shadow Binarization:** Standard binarization turned shaded areas of the paper entirely black, obscuring the text. This required abandoning global thresholds in favor of tuning block sizes for adaptive Gaussian thresholding.

11. Learnings & Key Takeaways

1. **Computer Vision Pipelines:** Gained practical experience in chaining low-level image processing techniques to solve high-level geometric problems.
2. **Matrix Mathematics in Images:** Developed a strong understanding of how images are represented as NumPy arrays and how spatial transformations (like homography) manipulate these matrices in real-time.
3. **Modular Software Design:** Learned the importance of decoupling code into specific functional files (`preprocess.py`, `transform.py`) rather than writing a single monolithic script, making debugging significantly easier.

12. Future Enhancements

1. **Machine Learning Segmentation:** Replacing the classical Canny edge detector with a lightweight ML model (e.g., MobileNet) to perfectly segment documents even when background contrast is extremely low.
2. **Batch Processing:** Adding a CLI flag to accept a full directory of images, processing them sequentially, and stitching them into a single multi-page PDF document.
3. **Optical Character Recognition (OCR):** Integrating `Tesseract-OCR` to automatically extract the text from the scanned image, allowing the generated PDF to be fully searchable and editable.
4. **Graphical User Interface (GUI):** Developing a lightweight web-based interface using Flask to allow non-technical users to upload and scan documents via a browser.

13. References

1. OpenCV Documentation: *Perspective Transformation and Image Warping*. Available at: <https://docs.opencv.org/>
2. Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc.
3. NumPy Official Documentation: *Vectorized operations and linear algebra*. Available at: <https://numpy.org/doc/>
4. Python Software Foundation. *Python 3.11 Standard Library*. Available at: <https://docs.python.org/3.11/>